School: ..... Campus: .....

Academic Year: ..... Subject Name: ..... Subject Code: .....

Semester: ..... Program: ..... Branch: ..... Specialization: .....

Date: .....

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiement : Solidity Pro – Advanced Patterns and Inheritance

Objective/Aim:

To study advanced Solidity programming concepts including inheritance and design patterns used in smart contract development.

Apparatus/Software Used:

- Computer/Laptop
- Internet connection
- Remix IDE (Solidity compiler)
- Web browser

Theory/Concept:

1. **Solidity Inheritance:**
   Inheritance in Solidity allows one smart contract to use the properties and functions of another contract. This helps in code reuse and better organization. A child contract can inherit from a parent contract using the is keyword. Solidity supports single and multiple inheritance, making contracts more modular and efficient.
2. **Advanced Patterns in Solidity:**
   Solidity uses common programming patterns like Ownable (for access control), Factory (to create contracts), and Modifier patterns (to restrict function access). These patterns improve security, readability, and maintainability of smart contracts. They are widely used in real-world blockchain applications to manage permissions and contract interactions.

Procedure:

1. Open Remix IDE in a web browser.
2. Create a new Solidity file.
3. Write a base (parent) contract with basic functions.
4. Create a child contract using inheritance.
5. Implement modifiers and advanced patterns like Ownable.
6. Compile the contract using Solidity compiler.
7. Deploy the contract on Remix environment.
8. Test the inherited functions and observe outputs.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// Parent Contract (Ownable Pattern)
contract Ownable {
    address public owner;

    constructor() {    72361 gas 48000 gas
        owner = msg.sender;
    }

    // Modifier to restrict access
    modifier onlyOwner() {
        require(msg.sender == owner, "Not the owner");
        _;
    }
}

// Child Contract (Inheritance)
contract MyContract is Ownable {

    uint public value;

    // Only owner can set value
    function setValue(uint _value) public onlyOwner {    24705 gas
```


```
    // Only owner can set value
    function setValue(uint _value) public onlyOwner {    24705 gas
```


```
        value = _value;
    }

    // Anyone can view value
    function getValue() public view returns (uint) {    2410 gas
        return value;
    }
}
```

Observation Table:

Concept Used	Description	Result
Inheritance	Parent-child contract relationship	Functions reused
Modifier Pattern	Restricts function access	Improved security
Ownable Pattern	Owner-based control	Controlled execution
Deployment	Contract deployed in Remix	Successful execution

ASSESMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/ Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	**50**		